

Εθνικό Μετσόβιο Πολυτεχνείο



Εργαστήριο Μικροϋπολογιστών

8η Εργαστηριακή Άσκηση

Χριστόφορος Χαραλάμπους 03121614

Φίλιππος Γιαννακόπουλος 03121629

Ομάδα 17

Ζήτημα 8.3

Το Ζήτημα 8.1 βρίσκεται στο αρχείο erg8.1.c. Το Ζήτημα 8.2 βρίσκεται στο αρχείο erg8.2.c. Το Ζήτημα 8.3 βρίσκεται στο αρχείο erg8.3.c.

Για σκοπούς απλότητας, παρουσιάζεται μόνο το Ζήτημα 8.3.

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <avr/interrupt.h>

#define UBRR_VALUE 103
#define URL "http://192.168.1.250:5000/data"

#define PCA9555_0_ADDRESS 0x40 // A0=A1=A2=0 by hardware
#define TWI_READ 1 // reading from twi device
#define TWI_WRITE 0 // writing to twi device
#define SCL_CLOCK 100000L // twi clock in Hz

//Fsc1=Fcpu/(16+2*TWBR0_VALUE*PRESCALER_VALUE)
#define TWBR0_VALUE ((F_CPU/SCL_CLOCK)-16)/2

#define TEMP_OFFSET 9.0

char buffer[50];
uint8_t buffer_pointer = 0;

const char success[] = "\"Success\"";

void init_buffer() {
    buffer_pointer = 0;
}

void write_buffer(char c) {
    buffer[buffer_pointer] = c;
    buffer_pointer++;

    if (buffer_pointer == 50) buffer_pointer = 0;
}

void display_buffer() {
    for (uint8_t i = 0; i < buffer_pointer; i++) {
        lcd_data(buffer[i]);
    }
}

uint8_t success_fail_buffer() {
    if (buffer_pointer < 2) return 0;
    for (uint8_t i = 0; i < 2; i++) {
        if (buffer[i] != success[i]) {
            return 0;
        }
    }
    return 1;
}

void usart_init(uint16_t ubrr) {
    UCSR0A = 0;
    UCSR0B = (1<<RXEN0) | (1<<TXEN0);
    UBRR0H = (unsigned char) (ubrr>>8);
    UBRR0L = (unsigned char) ubrr;
}
```

```

    UCSR0C = (3 << UCSZ00);
}

void usart_transmit(uint8_t data) {
    while (!(UCSR0A & (1<<UDRE0)));
    UDR0 = data;
}

char usart_receive() {
    while (!(UCSR0A & (1<<RXC0)));
    return UDR0;
}

// Transmits character array up until it finds a \n (which is sent) or a \0
void transmit_string(char arr[]) {
    uint8_t i = 0;
    while (arr[i] != '\n' && arr[i] != '\0') {
        usart_transmit(arr[i]);
        i++;
    }
    if (arr[i] != '\0') usart_transmit(arr[i]);
}

uint8_t receive_success_fail() {
    init_buffer();
    char letter = usart_receive();
    while (letter != '\n') {
        write_buffer(letter);
        letter = usart_receive();
    }
    return success_fail_buffer();
}

// PCA9555 REGISTERS
typedef enum {
    REG_INPUT_0 = 0,
    REG_INPUT_1 = 1,
    REG_OUTPUT_0 = 2,
    REG_OUTPUT_1 = 3,
    REG_POLARITY_INV_0 = 4,
    REG_POLARITY_INV_1 = 5,
    REG_CONFIGURATION_0 = 6,
    REG_CONFIGURATION_1 = 7
} PCA9555_REGISTERS;

//----- Master Transmitter/Receiver -----
#define TW_START 0x08
#define TW_REP_START 0x10
//----- Master Transmitter -----
#define TW_MT_SLA_ACK 0x18
#define TW_MT_SLA_NACK 0x20
#define TW_MT_DATA_ACK 0x28
//----- Master Receiver -----
#define TW_MR_SLA_ACK 0x40
#define TW_MR_SLA_NACK 0x48
#define TW_MR_DATA_NACK 0x58

#define TW_STATUS_MASK 0b1111000
#define TW_STATUS (TWSR0 & TW_STATUS_MASK)

uint8_t PREVIOUS = 0;

```

```

// Initializes TWI clock
void twi_init() {
    TWSR0 = 0;           // Prescaler = 1
    TWBR0 = TWBR0_VALUE; // SCL clock 100kHz
}

// Read one byte from twi device (while requesting more)
unsigned char twi_readAck() {
    TWCRO = (1<<TWINT) | (1<<TWEN) | (1<<TWEA);
    while (!(TWCRO & (1<<TWINT)));

    return TWDRO;
}

// Read one byte from twi device (followed by a stop condition)
unsigned char twi_readNak() {
    TWCRO = (1<<TWINT) | (1<<TWEN);
    while (!(TWCRO & (1<<TWINT)));

    return TWDRO;
}

// Sends start condition, address and transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_start(unsigned char address) {
    uint8_t twi_status;

    // Send START
    TWCRO = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

    // Wait till START transmission is completed
    while (!(TWCRO & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_START) && (twi_status != TW_REP_START))
        return 1;

    // Send device address
    TWDRO = address;
    TWCRO = (1<<TWINT) | (1<<TWEN);

    // Wait till address transmission is completed and ACK/NACK has been received.
    while (!(TWCRO & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK))
        return 1;

    return 0;
}

// Sends start condition, address and transfer direction
// Waits until device is ready
void twi_start_wait(unsigned char address) {
    uint8_t twi_status;

    while (1) {
        // Send START

```

```

    TWCR0 = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);

    // Wait till START transmission is completed
    while (!(TWCR0 & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_START) && (twi_status != TW_REP_START))
        continue;

    // Send device address
    TWDR0 = address;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // Wait till address transmission is completed and ACK/NACK has been received
    while (!(TWCR0 & (1<<TWINT)));

    // Check TWI Status
    twi_status = TW_STATUS & 0xF8;
    if ((twi_status != TW_MT_SLA_ACK) && (twi_status != TW_MR_SLA_ACK)) {
        // Device is busy, send STOP condition
        TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

        // Wait till STOP condition is executed and bus released
        while (!(TWCR0 & (1<<TWSTO)));

        continue;
    }

    break;
}

// Send one byte to the TWI device
// Returns 0 for write success, 1 for write failure
unsigned char twi_write(unsigned char data) {
    // Send data to the (already addressed) device
    TWDR0 = data;
    TWCR0 = (1<<TWINT) | (1<<TWEN);

    // Wait till transmission finishes
    while (!(TWCR0 & (1<<TWINT)));

    if ((TW_STATUS & 0xF8) != TW_MT_DATA_ACK)
        return 1;
    return 0;
}

// Send repeated start condition, address, transfer direction
// Returns 0 for success, 1 for failure
unsigned char twi_rep_start(unsigned char address) {
    return twi_start(address);
}

// Stops data transfer and releases TWI bus
void twi_stop() {
    // Send STOP condition
    TWCR0 = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);

    // Wait till STOP condition is executed and bus released
    while (!(TWCR0 & (1<<TWSTO)));
}

```

```

}

// PCA9555 functions

void PCA9555_0_write(PCA9555_REGISTERS reg, uint8_t value) {
    PREVIOUS = value;
    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_write(value);
    twi_stop();
}

uint8_t PCA9555_0_read(PCA9555_REGISTERS reg) {
    uint8_t ret_val;

    twi_start_wait(PCA9555_0_ADDRESS + TWI_WRITE);
    twi_write(reg);
    twi_rep_start(PCA9555_0_ADDRESS + TWI_READ);
    ret_val = twi_readNak();
    twi_stop();

    return ret_val;
}

void write2(unsigned char input) {
    unsigned char prev = PREVIOUS;

    unsigned char write = (input & 0xF0) | (prev & 0x0F);
    PCA9555_0_write(REG_OUTPUT_0, write);

    write |= (1<<3);
    PCA9555_0_write(REG_OUTPUT_0, write);
    write &= 0b11110111;
    PCA9555_0_write(REG_OUTPUT_0, write);

    write = ((input & 0x0F) << 4) | (prev & 0x0F);
    PCA9555_0_write(REG_OUTPUT_0, write);

    write |= (1<<3);
    PCA9555_0_write(REG_OUTPUT_0, write);
    write &= 0b11110111;
    PCA9555_0_write(REG_OUTPUT_0, write);
}

void lcd_data(unsigned char input) {
    unsigned char prev = PREVIOUS;
    prev |= (1<<2);

    PCA9555_0_write(REG_OUTPUT_0, prev);
    write2(input);
    _delay_us(250);
}

void lcd_command(unsigned char input) {
    unsigned char prev = PREVIOUS;
    prev &= 0b11111011;

    PCA9555_0_write(REG_OUTPUT_0, prev);
    write2(input);
    _delay_us(250);
}

```

```

void lcd_nextline() {
    lcd_command(0b11000000);
}

void lcd_clear() {
    lcd_command(0x01);
    _delay_ms(5);
}

void lcd_init() {
    _delay_ms(200);

    // Switch to 8bit mode
    unsigned char write = 0x30;
    PCA9555_0_write(REG_OUTPUT_0, write);

    unsigned char temp = write;

    temp |= (1<<3);
    PCA9555_0_write(REG_OUTPUT_0, temp);
    temp &= 0b11110111;
    PCA9555_0_write(REG_OUTPUT_0, temp);
    _delay_us(250);

    temp = write;

    // Switch to 8bit mode
    PCA9555_0_write(REG_OUTPUT_0, write);

    temp |= (1<<3);
    PCA9555_0_write(REG_OUTPUT_0, temp);
    temp &= 0b11110111;
    PCA9555_0_write(REG_OUTPUT_0, temp);
    _delay_us(250);

    temp = write;

    // Switch to 8bit mode
    PCA9555_0_write(REG_OUTPUT_0, write);

    temp |= (1<<3);
    PCA9555_0_write(REG_OUTPUT_0, temp);
    temp &= 0b11110111;
    PCA9555_0_write(REG_OUTPUT_0, temp);
    _delay_us(250);

    // Switch to 8bit mode
    write = 0x20;
    temp = write;
    PCA9555_0_write(REG_OUTPUT_0, write);

    temp |= (1<<3);
    PCA9555_0_write(REG_OUTPUT_0, temp);
    temp &= 0b11110111;
    PCA9555_0_write(REG_OUTPUT_0, temp);
    _delay_us(250);

    lcd_command(0x28);

    lcd_command(0x0c);
}

```

```

    lcd_clear();

    lcd_command(0x06);
}

void display_number(uint8_t input) {
    lcd_data(0b00110000 | (input & 0x0F));
}

// Displays character array up until it finds a \0
void display_string(char arr[]) {
    uint8_t i = 0;
    while (arr[i] != '\0') {
        lcd_data(arr[i]);
        i++;
    }
}

// Temperature Functions
// Returns 1 if device is detected, 0 if no.
uint8_t one_wire_reset() {
    DDRD |= 1<<4;
    PORTD = 0;

    _delay_us(480);

    DDRD = 0;
    PORTD = 0;

    _delay_us(100);

    uint8_t temp = PIND;

    _delay_us(380);

    return (temp & 1<<4) ? 0 : 1;
}

uint8_t one_wire_receive_bit() {
    DDRD |= 1<<4;
    PORTD = 0;

    _delay_us(2);

    DDRD = 0;
    PORTD = 0;

    _delay_us(10);

    uint8_t temp = (PIND & 1<<4) ? 1 : 0;

    _delay_us(49);

    return temp;
}

void one_wire_transmit_bit(uint8_t bit) {
    DDRD |= 1<<4;
    PORTD = 0;

```



```

    _delay_us(2);

    PORTD = bit<<4;

    _delay_us(58);

    DDRD = 0;
    PORTD = 0;

    _delay_us(1);
}

uint8_t one_wire_receive_byte() {
    uint8_t output = 0x00;

    for (uint8_t i = 0; i < 8; i++) {
        output = output>>1;
        if (one_wire_receive_bit()) {
            output |= 1<<7;
        }
    }

    return output;
}

void one_wire_transmit_byte(uint8_t byte) {
    uint8_t temp = 1;
    for (uint8_t i = 0; i < 8; i++) {
        if (byte & temp) {
            one_wire_transmit_bit(1);
        } else {
            one_wire_transmit_bit(0);
        }
        temp = temp << 1;
    }
}

uint16_t temp() {
    if (!one_wire_reset()) {
        return 0x8000;
    }

    one_wire_transmit_byte(0xCC);

    one_wire_transmit_byte(0x44);

    while (!one_wire_receive_bit());

    if (!one_wire_reset()) {
        return 0x8000;
    }

    one_wire_transmit_byte(0xCC);

    one_wire_transmit_byte(0xBE);

    uint16_t output = one_wire_receive_byte();
    output |= one_wire_receive_byte()<<8;

    return output;
}

```

```

double get_temp() {
    uint16_t reading = temp();
    if (reading == 0x8000) return 0;

    int16_t val = reading;
    double temperature_val = val / 16.0;

    return temperature_val + TEMP_OFFSET;
}

// Keypad functions
const uint8_t rows[] = {0b11111110, 0b11111101, 0b11111011, 0b11110111};
const char characters[] = {'1','2','3','A','4','5','6','B','7','8','9','C','*','0','#','D'};
uint16_t pressed_keys = 0x0000;

uint8_t scan_row(uint8_t row) {
    PCA9555_0_write(REG_OUTPUT_1, rows[row]);
    uint8_t input = PCA9555_0_read(REG_INPUT_1);
    input = ~input;
    return input>>4;
}

uint16_t scan_keypad() {
    uint16_t output = 0x0000;
    uint8_t temp;
    for (uint8_t i = 0; i < 4; i++) {
        output = output<<4;
        temp = scan_row(i) & 0x0F;
        output |= temp;
    }

    return output;
}

void scan_keypad_rising_edge() {
    uint16_t pressed_keys_tempo = scan_keypad();
    _delay_ms(10);
    pressed_keys_tempo &= scan_keypad();

    pressed_keys = pressed_keys_tempo & (~pressed_keys);
}

char keypad_to_ascii(uint16_t keys) {
    uint16_t test = 1;
    for (uint8_t i = 0; i < 16; i++) {
        if (keys & test) {
            return characters[i];
        }
        test = test<<1;
    }
    return '\0';
}

double get_pressure() {
    ADCSRA |= (1<<ADSC);
    while ((ADCSRA & (1<<ADSC)) != 0);

    // Processing
    double pressure = ADC * 20;
    pressure = pressure / 1024;
}

```

```

    return pressure;
}

unsigned char thousands, hundreds, tens, units;

void binary_to_bcd(uint16_t input) {
    thousands = input / 1000;
    input = input % 1000;

    hundreds = input / 100;
    input = input % 100;

    tens = input / 10;
    units = input % 10;
}

void transmit_temp(double input) {
    uint8_t negative_sign = (input < 0) ? 1 : 0;

    if (negative_sign) input = -input;

    uint16_t integer = input;
    uint16_t decimal = (input - integer) * 10000;

    binary_to_bcd(integer);

    if (negative_sign) {
        uart_transmit('-');
    }

    if (hundreds != 0) uart_transmit(hundreds + '0');
    if (tens != 0 || hundreds != 0) uart_transmit(tens + '0');
    uart_transmit(units + '0');
    uart_transmit('.');

    binary_to_bcd(decimal);
    uart_transmit(units + '0');
}

void transmit_pressure(double input) {
    uint16_t integer = input;
    uint16_t decimal = (input - integer) * 10000;

    binary_to_bcd(integer);

    if (tens != 0) uart_transmit(tens + '0');
    uart_transmit(units + '0');
    uart_transmit('.');

    binary_to_bcd(decimal);
    uart_transmit(thousands + '0');
}

int main(void) {
    // Enable TWI Communication for LCD
    twi_init();

    // Configure EXT_PORT1 as output
    PCA9555_0_write(REG_CONFIGURATION_0, 0x00);
    PCA9555_0_write(REG_CONFIGURATION_1, 0b11110000);
}

```

```

// Enable LCD
lcd_init();

init_buffer();

// POT0 ADC Setup
ADMUX = (1<<REFS0);
ADCSRA = (1<<ADEN) | (1<<ADPS2) | (1<<ADPS1) | (1<<ADPS0);

usart_init(UBRR_VALUE);

uint8_t nurseStatus = 0;

transmit_string("ESP:restart\n");
char letter = usart_receive();
while (letter != '\n') {
    letter = usart_receive();
}
letter = usart_receive();
while (letter != '\n') {
    letter = usart_receive();
}

while (1) {
    lcd_clear();
    transmit_string("ESP:connect\n");

    if (!receive_success_fail()) {
        display_string("1.Fail");
        lcd_nextline();
        display_string("Rec:");
        display_buffer();
        _delay_ms(1000);
        transmit_string("ESP:connect\n");
        if (receive_success_fail()) {
            lcd_clear();
        } else {
            continue;
        }
    }
    display_string("1.Success");
    lcd_nextline();
    display_string("Rec:");
    display_buffer();

    _delay_ms(1000);

    transmit_string("ESP:url:\n");

    transmit_string(URL);

    transmit_string("\n\n");

    lcd_clear();

    if (receive_success_fail()) {
        display_string("2.Success");
    } else {
        display_string("2.Fail");
        lcd_nextline();
    }
}

```

```

        display_string("Rec:");
        display_buffer();
        _delay_ms(1000);
        continue;
    }

    lcd_nextline();
    display_string("Rec:");
    display_buffer();

    _delay_ms(1000);

    double temp = get_temp();
    double pressure = get_pressure();

    char pressed_char = keypad_to_ascii(scan_keypad());
    if (pressed_char == '7') {
        nurseStatus = 1;
    } else if (pressed_char == '#') {
        nurseStatus = 0;
    }

    transmit_string("ESP:payload:[{\"name\": \"temperature\", \"value\": \"\"});

    transmit_temp(temp);

    transmit_string("\",{\"name\": \"pressure\", \"value\": \"\"});

    transmit_pressure(pressure);

    transmit_string("\",{\"name\": \"team\", \"value\": \"17\"},{\"name\": \"status\",
    \"value\": \"\"});

    if (nurseStatus) {
        transmit_string("NURSE CALL");
    } else if (pressure > 12 || pressure < 4) {
        transmit_string("CHECK PRESSURE");
    } else if (temp > 37 || temp < 34) {
        transmit_string("CHECK TEMP");
    } else {
        transmit_string("OK");
    }

    transmit_string("\"]]\n");

    lcd_clear();

    if (receive_success_fail()) {
        display_string("3.Success");
    } else {
        display_string("3.Fail");
        _delay_ms(1000);
        continue;
    }

    lcd_nextline();

    display_string("Rec:");
    display_buffer();

    _delay_ms(1000);

```

```
transmit_string("ESP:transmit\n");

lcd_clear();

display_string("4.Success");
lcd_nextline();
init_buffer();

char readChar = usart_receive();
while (readChar != '\n') {
    write_buffer(readChar);
    readChar = usart_receive();
}

display_buffer();

_delay_ms(5000);
}
}
```